Scientific Reports

https://doi.org/10.1038/s41598-026-57474-6

Article in Press

A self-orchestrating physics-discovering neural architecture for adaptive and fault-resilient robotic motion control via RPA and digital twins

Received: 24 February 2026

Accepted: 8 June 2026

Published online: 22 June 2026

Cite this article as: Singla D., Logeswari G., Tamilarasi K. *et al.* A self-orchestrating physics-discovering neural architecture for adaptive and fault-resilient robotic motion control via RPA and digital twins. *Sci Rep* (2026). https://doi.org/10.1038/s41598-026-57474-6

Daksh Singla, G. Logeswari, K. Tamilarasi & J. Deepika Roselind

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

ARTICLE IN PRESS

A Self-Orchestrating Physics-Discovering Neural Architecture for Adaptive and Fault-Resilient Robotic Motion Control via RPA and Digital Twins

Daksh Singla[1], G. Logeswari[1,*] , K. Tamilarasi[2], J. Deepika Roselind[2]
[1,2]Vellore Institute of Technology, Chennai, Tamil Nadu, India
[*]Corresponding author: Logeswari G (e-mail: logeswari.g@vit.ac.in)

ABSTRACT To autonomously control a robot's movement in complex and unpredictable environments, adaptive models must be developed that can continuously learn about and refine their future physical dynamics. This paper introduces a Self-Organising Physics Discovered Neural Architecture (SOPDNA) that unites Physics-Informed Neural Networks (PINNs) with Robotics Process Automation (RPA) and Digital Twin DT technologies to create an entirely automated, robust robotic motion control system. In contrast to traditional robotic motion control systems that rely on predefined mathematical models or offline training methods, SOPDNA automatically discovers previously unknown physical relationships between robotic motion components and continuously updates the controller policy during operation. The Digital Twin software acts as a nearly identical simulation environment to enable extensive self-experimentation and safety verification of SOPDNA while the RPA software automates all aspects of the SOPDNA learning process, including acquiring experimental data, detecting anomalies, retraining the learning model, and deploying optimised controllers, without requiring any human oversight. The core of SOPDNA consists of a PINN that incorporates a comprehensive mathematical representation of the robot's physical constraints and uses symbolic regression techniques to estimate unmodelled dynamic variables, such as variable friction coefficients and environmental disturbances. Experimental validation using a robotic manipulator performing trajectory tracking and obstacle avoidance under changing payloads shows the proposed approach's capabilities. The SOPDNA framework provides an overall trajectory-tracking accuracy of 98.4%, a 22.7% reduction in energy usage, and a 30.9% improvement in disturbance rejection when compared to traditional deep reinforcement learning (DRL) and Proportional-Integral-Derivative (PID)-based control systems. Results from the fault-injection experiments show SOPDNA's ability to recover from degradation of actuators 41.3% faster than conventional methods. Overall, these findings support the potential for self-organising physics-discovered neural network architectures to enhance the development of robust, adaptive, intelligent robotic motion control systems capable of autonomously executing complex tasks.

INDEX TERMS Physics-Informed Neural Networks, Robotic Process Automation, Digital Twin, Autonomous Robotic Motion Control, Physics Discovery, Self-Orchestrating Learning Systems, Fault-Resilient Robotics.

I. INTRODUCTION

The emergence of autonomous robotic systems has revolutionized technology today through their implementation across many industries including manufacturing, healthcare, logistics, agriculture, and exploratory domains. In order for these machines to perform effectively in dynamic and unpredictable or unknown situations, they must be able to navigate accurately, stably and adaptively to continuously changing physical conditions; this is referred to as the problem of robotic motion control. The greater part of motion control is how to achieve these motions (move along specific paths) with respect to an environment using traditional control architecture-based solutions such as PID controller schemes or model-based optimal

ARTICLE IN PRESS

controllers generally use detailed mathematical models of robot dynamics; however, real-world applications seldom meet these criteria because there are never complete physical models available since real world scenarios can vary greatly due to factors like changes in friction, payloads, sensor noise, mechanical wear and unrestrained interactions with the environment and therefore result in performance degradation and limit robustness.

The recent advances in Machine Learning (ML) provide new opportunities to look at the problem of control using a different perspective where the control system learns through observation rather than pure exploration of the robot's environment. Many researchers have begun to explore the use of DRL as a method for developing intelligent robot systems or neural network based control techniques in both a simulated or controlled manner that are capable of providing excellent performance on controlled systems. However, due to their nature as black-box models and their need for highly extensive amounts of training data to be successful, the costs associated with obtaining this training data or physical implementation on actual robots may not yield a productive return. In addition, ML-based systems lack one of the most fundamental properties; they do not adhere to any physical principles within the realm of physics and therefore do not have applicable generalization when interacting with an environment that is outside of the distribution of the training experience data. This disconnection between ML-based techniques and the basic foundations of how to operate efficiently within the bounds of the laws of physics indicate that there is a need to explore hybrid approaches to robotic control that will leverage both physical knowledge/domain knowledge and data driven intelligence [1].

PINNs have become an excellent way to connect the gap between data-driven models and their corresponding physical laws; incorporating governing equations, directly into the training of the neural network, allows the constraints of the learning objective, stated by the differential equations, of the behaviours of the system, to create both data-efficient models, and physically accurate models [2]. PINNs have been applied within robotic motion control for tasks like inverse dynamics approximation, trajectory generation and state estimation, all with adherence to kinematic and dynamic constraints. However, the majority of existing robotic PINN frameworks have been founded under the assumption that the physics governing the underlying system is well-defined and static. In reality, robots operate under a variety of real-world environments where they can experience unknown or time-varying aspects of the physical model (e.g. contact dynamics, friction coefficients, external disturbances) - something that is a limitation to the adaptability of typical PINN approaches and prevents long-term autonomous operation within complex environments.

ARTICLE IN PRESS

Another challenge in current robotic learning systems; relies heavily on human engineering providing manual intervention at multiple points within the training and deployment pipeline. During this time, human engineers are responsible for data collection, retraining of models, hyperparameter tuning for all models, performance monitoring for all models, and updating controller models. The result of relying on human supervision for these tasks has a diminishing effect on scalability and makes real-time adaptation extremely challenging. In order to develop fully autonomous robotic intelligence, new systems will need to be developed that are capable of self-managing their own learning processes, determining failures, and refining control strategies without the need for continuous external supervision [3].

RPA, which is used primarily within businesses and software workflow automation, is a new avenue for automating repetitive, rule-based processes. RPA can serve as a method of automation for robotic systems using a level of orchestration to manage the following robotic functions: data collection (to create data sets for training models), triggering a model to retrain, validating performance of the static and dynamic robot controller, and facilitating the deployment of the updated controller. By placing rules for making decisions and policies to adapt to changing conditions within RPA, RPA agents can allow robotic systems to autonomously manage their own learning lifecycle, thereby making robotic systems self-orchestrating. There has been little research conducted in the field of robotics on the

ARTICLE IN PRESS

integration of RPA with learning-based control, though this combination can be an effective way to reduce the amount of human input for operations and increase the autonomy of robotic operations [4].

Digital Twin technology, on the other hand, has been used and gained momentum for the purpose of creating virtual copies of physical systems that are continuously synchronized in real-time with sensor data [5]. Digital twins provide an environment for experimentation to occur in a safe manner, performing predictive analytics and optimizing the system as if the physical robot is still intact. Specifically for robotic motion control purposes, a digital twin can develop different control strategies that simulate the reaction of the robot to a disturbance, and can assess the safety limits of the system prior to implementing those changes in the actual physical robot [6]. When digital twins and learning algorithms are combined, a digital twin can be used as a type of controlled environment for carrying out large-scale self-experimentation, and as a mechanism for continuous improvement. However, today's state-of-the-art digital twin applications for robotics mainly focus on monitoring and displaying the robotic system, rather than facilitating autonomous learning and physical phenomena discovery [7].

With the blending of PINNs, RPA, and Digital Twins, we are approaching a point where robots can not only be trained through data but can also learn physical phenomena and how to manage their own workflows, and in addition perform continuous in-situ and virtual testing of actions to validate themselves [8]. This new way of controlling robots is a new era because we will move away from static models of robot learning to self-evolving robotic technologies. Learning systems with the ability to discover unknown or un-modeled physical (or mathematical) relationships from observed data while still consistent with known laws are termed as discovering physical phenomena [9].

Robots in open world environments must be able to operate in unpredictable environments and discover physical interactions and conditions that were not previously considered [10]. While we have made significant advances in each of these areas individually, a unified architecture for controlling autonomous robots using all three is still in its infancy. Most studies conducted, focus on either physics based learning model or utilizing digital twin simulation or workflow automated, and not all three combined together into one cohesive self-orchestrating system. In addition to that fact that existing robotic control systems do not contain the capacity for online improvement of their physical models or automated fault or performance degradation recovery, many of our robotic systems continue to remain brittle and require constant human intervention to recalibrate themselves.

This paper presents a SOPDNA for autonomous robotic movement control based on integrating PINNs, RPA, and Digital Twin technologies. The proposed framework is designed to allow for continuous learning, validation, and refinement of robotic dynamic understanding while performing control tasks in real-time. The PINN component incorporates established physical constraints while concurrently discovering unknown dynamic terms by means of data-driven learning. The digital twin serves to provide a synchronized virtual space that will enable simulation-based validation and large-scale experimentations. The RPA layer orchestrates the entire process by automating data acquisition, anomaly detection; retraining schedules and control deployment. In contrast to traditional robotic learning systems that rely on offline training and manual updates, the proposed architecture establishes a closed-loop self-management cycle. Sensor data from the physical robot are send to the digital twin, which in turn evaluates candidate control strategies. When deviations or anomalies occur, RPA agents trigger the retraining of the PINN model with revised physical constraints and re-deploy the optimized controller back to the physical robot. This closed-loop continuous feedback system allows for the robot to adjust to changing conditions such as changes in payload, actuator degradation, or environmental disturbances, without requiring any explicit human intervention.

This research is motivated by the challenges faced by existing robotic motion control techniques in terms of fixed physical models, autonomous learning orchestration, and limited

ARTICLE IN PRESS

use of digital twinned approaches for adaptive control. In particular, we show how self-discovery through physics allows the system to change its internal representation of physics when physical environments change. Through self-orchestration, we can manage our own learning lifecycle. Additionally, we use a digital twin to explore and validate control approaches in a safe manner prior to executing those control strategies in the real world. Together, these three components provide a scalable and reliable foundation for next-generation autonomous robots.

This study contributes three main areas of knowledge. First, we provide a cohesive architecture that integrates physics-informed learning, automation of the training pipeline, and digital twin validation into one autonomous robotic motion control device. Second, we show that systems can discover and incorporate live unknown or time-changing physical dynamics in real-time into their control policy. Third, we create a self-orchestrating control loop that decreases needed human intervention and increases robustness to fault and disturbance conditions.

The organization of this article is as follows. Review of relevant literature concerning physics-based learning, robotic motion control, RPA, and digital twins will be covered in Section 2. The comprehensive technical details of the self-orchestrating physics-discovering architecture, including its components and learning process, will be presented in Section 3. Section 4 will describe the experimental setup, the metrics used for evaluating performance, and present the results of the experiments along with analyses of system performance under various conditions. Lastly, Section 5 concludes this paper and outlines future work toward achieving completely autonomous, self-learning robotics.

This research aims to develop a means for robotic motion control to evolve from static and model-dependent systems to intelligent agents that are able to learn and independently develop their own understanding of the physical world. This represents a very important progression toward achieving truly autonomous robots that can operate in complex and unpredictable environments for an extended period of time.

II. LITERATURE REVIEW

In recent years, there have been many advances in the area of autonomous robotic motion control and the shift towards using hybrid motion control frameworks, which are based on both the physical knowledge about how the motion of a robotic system occurs as well as modern data-driven learning approaches. Traditionally, robotic control has used methods, like PID controllers, sliding mode control, and model predictive control, that require a precise analytic model for the dynamics of the robot [11]. However, most robotic systems operate in environments that have elements of uncertainty, unmodeled dynamics, sensor noise, and external disturbances. This has resulted in researchers investigating the use of a combination of ways to model robotic systems through a combination of simulation and a physical-based modeling approach along with machine learning in a manner which integrates the physics-based modeling elements (i.e., laws governing the motion of a robotic system) directly into the neural learning structure of the robotic system in order to accomplish the goal of improved generalization, robustness, and adaptability.

A significant advancement in this area of study has been the development of PINNs, which is an approach to designing neural networks to model physical systems and to include the laws of physics into the training of the network instead of treating the neural network like a black box to approximate functions. An example would be embedding the governing physics equations (e.g., Newton-Euler dynamics or Lagrangian formulations) as part of the loss function for training the network so that the network is required to satisfy the physical constraints of the physical laws associated with the motion of a robotic system, while also minimizing the error between data-driven results and ground truth in a manner which usually requires significantly less labeled data than the traditional approach to developing deep learning networks [12]. PINN concepts were introduced by Raissi et al., who demonstrated the application of PINNs to solving forward and inverse problems for partial differential

ARTICLE IN PRESS

equations (PDEs) [13], providing a strong theoretical underpinning for the integration of neural learning with the constraints of differential equations.

Training on this basis, Karniadakis and coauthors provide a thorough overview of the rapidly developing field of physics-informed machine learning. They provide an overview of the techniques for combining traditional numerical solvers with deep learning methods in order to efficiently model complex dynamical systems [14]. They demonstrate that when you construct a neural network that embodies the underlying physical laws in its architecture, your network is capable of producing more generalized output, making it easier to interpret, and is less dependent on data as input than when using traditional numerical solvers. These important conclusions have since been implemented in the field of robotics to create models of nonlinear dynamics and improve control strategies.

A major breakthrough in the field of physics-informed modeling occurred with the introduction of Neural ODEs [15]. Neural ODEs can be viewed as flows through a neural network, providing a method to learn smooth trajectories of robot motion. Neural ODEs offer an alternative to discrete representations of data as they can learn time-variant behavior directly from the data. However, the use of continuous optimization for training introduces numerical instability and cost due to large computation requirements, which is a critical drawback for real-time control applications.

In addition, techniques for discovering sparse models have also impacted the field of robotics. One example is the development of Sparse Identification of Nonlinear Dynamics (SINDy) by Brunton and coauthors, which is a method that provides a way to extract minimal governing equations for nonlinear systems from time series data using sparse regression [16]. Although originally applied outside of the robotics domain, SINDy has been implemented to learn interpretable dynamic models of robotic systems, especially when the governing physics are partially unknown or latent in sensor data.

The use of PINNs has been employed in robotics to create models and control variables across multiple systems, including Liu et al., who extended the use of PINNs for robotic modeling by training networks with rigid body dynamics embedded into the training of the network to achieve higher accuracy in trajectory predictions and control of both rigid and soft systems [17]. Their findings demonstrate that physics-informed learning is able to model complex interactions, including friction and compliance, which are difficult to analytically model.

Li et al. have developed a PINN Predictive Control (PINNPC) framework for quadrupedal locomotion which includes the ability to perform on-line payload estimation directly into the PINN model [18]. The authors had the capability to couple PINNs with nonlinear Model Predictive Control, leading to a PINNPC framework that is able to adapt to variable loads and to adapt to interaction with the environment to achieve robust real-time gait control. Furthermore, this work demonstrates the advantages of using physics-informed models in an adaptive control loop.

Compliant robotic systems, such as soft and flexible or elastic jointed robots, present an additional challenge to utilize physics-informed learning as a modelling tool because of the deformability characteristics of these types of robots. Li et al. have developed a PINN model that represents the compliant manipulator as a compliant manipulator and captures both the rigid and elastic dynamical characteristics of the manipulator, allowing for accurate dynamic prediction and control in the presence of uncertainty [19]. Their results provide evidence of the value of using physics-informed learning in systems that do not have the ability to model traditional rigid dynamics.

In addition to PINN formulations based on physics-based principles, there are also various approaches that combine physical guidance and deep reinforcement learning (RL). For example, researchers, including Banerjee et al., have discussed how Physics-Informed Reinforcement Learning (PIRL) employs physical constraints incorporated into either the reward or the policy structure in order to improve sample efficiency and both convergence

ARTICLE IN PRESS

rates [20]. Due to balancing exploration against consistency with the physical laws governing the underlying system, PIRL enables autonomous robots to learn control policies that respect the known dynamics of their environment.

Another area of study involves new transformer-based neural architectures applied to improve performance, such as that introduced by Golroudbari et al., using a transformer-based PINN that uses attention mechanisms to provide long-term orientation and state estimate capabilities in robotic systems [21]. Their architecture demonstrated superior performance relative to both traditional LSTM and EKF methods in the presence of highly dynamic environments, indicating the possible advantage of combining structured sequence models with physical constraints.

While much of the work on PINNs and PIRL has focused on modeling and control, they may be viewed as complementary paradigms to the concept of the digital twin, which uses a high fidelity digital representation of the robot. Digital twins are able to simulate the dynamics of a robot in real time, providing a foundation for validation, prediction, and optimization capabilities. The concept of digital twins was popularized by Tao et al. for use in manufacturing applications, including their role in the monitoring and control of cyber-physical systems [22]. Grieves and Vickers subsequently defined the digital twin as a synchronized virtual and physical counterpart which allows for decision making and predictive analytics through the use of the digital twin [23].

Digital twins have been utilized for sim-to-real transfer in the robotics domain. For example, Mohaghegh et al. introduced a digital twin testbed that enables the deployment of imitation learning in mobile robots without any need for retraining in real world [24]. Their system allows for zero-shot transfer of learned policies from simulated environments, thanks to the close synchronization of physical sensors and simulations. Similarly, Liu et al. integrated digital twins with reinforcement learning for robotic manipulation, using virtual feedback to fine-tune control policies prior to physical deployment [25]. These contributions demonstrate the potential of digital twins to mitigate the reality gap and enhance controller reliability.

In addition, multiple authors have discussed the utility of digital twin frameworks for fault diagnosis and maintenance. Wang et al. presented a hybrid autoencoder-simulator design for detecting real-time anomalies in industrial robots. Their model uses digital twin data for identifying deviations from expected performance [26]. Thus, their hybrid approach allows for early detection of mechanical faults, improving reliability and uptime.

In addition to detecting hardware faults, research has also investigated the use of physics-informed models in conjunction with digital twin systems. Datta et al. proposed physics-encoded neural networks in the context of digital twin modeling, whereby the inclusion of physical prior knowledge within a neural network results in improved generalization performance and computational efficiency relative to traditional approaches [27]. Overall, their hybrid architecture outperformed traditional data-driven digital twins in terms of performance, especially in situations where physical constraints are important.

Physics-informed learning has also influenced robotic motion planning. For instance, researchers [28] have demonstrated that by employing PINNs to plan trajectories for heavy machinery (i.e., excavators), there is less deviation from optimal trajectories at a reduced computational cost. Ni et al. [29] extended this work by incorporating physics into the planning of motions on manifolds, thereby developing planning algorithms capable of dealing with high degree-of-freedom robots.

There have been few studies that have directly addressed self-orchestrating architectures that encompass physics discovery, digital twin simulation, automated retraining, and deployment as an integrated system. Most current research has been conducted around either physics-based modelling alone or digital twin supported control systems, however the proposed SOPDNA aims to integrate these components by introducing an automated orchestration engine which can support autonomous learning, validation, and deployment in

ARTICLE IN PRESS

a closed loop. A comparative analysis of existing physics-informed and hybrid learning approaches to robotic motion control is provided as Table 1.

Table 1. Summary of Related Work and Their Relevance to the Proposed Framework

Ref	Approach	Learning Model	Physics Integration	Digital Twin	Orchestration	Control Type	Dataset / Environment	Performance Metrics	Strengths	Limitations
[3]	PINN	PINN	PDE constraints	No	No	Model & inverses	Synthetic & PDE tests	Loss, residual	Data-efficient	Computational cost
[6]	SINDy	Sparse regression	Equation discovery	No	No	System ID	Time series	RMSE	Interpretability	Sensitive to noise
[4]	Physics-ML	Hybrid DNN	Latent physics	No	No	Prediction	Sim data	Error bounds	Unified theory	Training complexity
[5]	Neural ODE	ODE-NN	Continuous dynamics	No	No	Trajectory	Simulated motion	Smoothness	Realistic modeling	Numerical instability
[7]	PINN + Robot	PINN	Known robot dynamics	No	No	Trajectory & control	Robot testbeds	Tracking error	Physical consistency	Needs accurate physics
[8]	PINNPC	PINN + MPC	Payload dynamics	No	Yes	Locomotion control	Quadruped	RMSE, stability	Adaptive control	Model complexity
[9]	PINN compliant	PINN	Elastic dynamics	No	No	Manipulator	Flexible robots	Accuracy	Captures compliance	Data-intensive
[10]	PIRL survey	RL + physics	Physics in RL	No	No	RL control	Benchmarks	Convergence	Sample efficiency	Simulation bias
[11]	Transformer PINN	PINN + Transformer	Physics residuals	No	No	State estimation	High-dim data	MAE	Long-term prediction	Data requirement
[12]	Digital Twin overview	N/A	Simulation based	Yes	No	Monitoring & control	Industrial	Response time	Real-time sync	Sensor noise
[13]	Digital Twin concept	N/A	Virtual replica	Yes	No	System modeling	CPS	Fidelity	Unified framework	Implementation cost
[14]	DT sim-to-real	IL + DT	Synced sensors	Yes	No	Sim-to-Real	Mobile robots	Transfer success	Zero-shot transfer	Simulation gap
[15]	RL + DT	RL	Simulation adaptation	Yes	No	Manipulation	Sim + real	Reward	Sim gap reduced	Transfer bias
[16]	DT anomaly	Hybrid AE + NN	Residual physics	Yes	Partial	Diagnostics	Industrial robots	Fault accuracy	Early detection	High computation
[17]	Physics inside DT	PINN + DT	Physics prior	Yes	No	Twin modeling	Dynamic systems	RMSE	Efficient learning	Twin overhead
[18]	PINN planning	PINN	Motion constraints	No	No	Motion planning	Excavators	Deviation	Reduced cost	Domain specific
[19]	PINN manifolds	PINN	Navigation constraints	No	No	High-DOF planning	Sim env	MAE	High accuracy	Large dataset

ARTICLE IN PRESS

ARTICLE IN PRESS

—	Self-Orchestrating	SOPDNA (this work)	Physics + discovery	Yes	Yes	Autonomous control	CPS robots	Adaptation rate	Full automation	System complexity

III. PROPOSED METHODOLOGY

The SOPDNA framework will provide autonomous robotic motion control through the collaborative connection of three major components: a Digital Twin, a Physics-Discovering Neural Network based on PINNs, and an RPA-driven orchestration engine. A closed-loop control and learning cycle is created where physical interactions of the robotic system are replicated continuously in a high-precision Digital Twin, while the neural system learns latent laws of physical motion via motion dynamics and updates the control algorithm real-time.

Operating through an iterative cycle of perception, learning, validation, and deployment, data from sensors on the actual robot will be streamed to the Digital Twin, generating the kinematic and dynamic behaviour of that robot in a simulated form across various operational conditions. The Physics-Discovering Neural Network model can learn both the trajectory of the states and unknown physical properties by enforcing physical constraints within the training of that model. The RPA layer will automatically handle the ingestion of the different forms of data (from real and simulated), creation of retraining schedules, failure detection, and updating of controllers without human intervention. This will allow the system to self-orchestrate its learning process and provide reliable autonomous motion control in dynamic and uncertain environments.

When connected together, these three components allow: (1) the creation of noise-aware training datasets which will enable the neural model to generate interpretable, physics-based representations of physical phenomena; (2) the RPA engine to provide real-time coordination of the continuously optimizing process; and (3) the overall scalability, adaptability, and resiliency of the SOPDNA framework. The architectural diagram of the proposed self-orchestrating robotic system is presented in Figure 1.

Digital Twin–Driven Data Acquisition and Synchronization Module

The Digital Twin module replicates the physical robotic system and generates and validates data for that system; it continuously synchronises with the physical robot using bi-directional data exchange to enable forward-ahead simulations and inversion validation of motion trajectories. Kinematic states (e.g. joint angles and velocities) as well as dynamic quantities (e.g. torque, acceleration, external disturbances) are captured by the digital twin.

The Digital Twin will be developed to mathematically model the motion of the robotic manipulator through the use of a generalized nonlinear dynamic equation that relates joint acceleration, applied torque, and external forces as given in Eq.(1).

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) + F_d = \tau \qquad (1)$$

where q is the vector of joint positions, $\dot{q}$ is the vector of joint velocities, and $\ddot{q}$ represent the vector of joint accelerations, $M(q)$ is the inertia matrix, $C(q,\dot{q})$ is the Coriolis and centrifugal matrix, $G(q)$ is the gravitational force vector caused by gravity, F_d accounts for external disturbances and frictional forces affecting motion, and τ denotes the applied torque about each joint.

The digital twin generates physically accurate training trajectories, reflecting the nonlinear full dynamics of the robot. Actuator noise, payload variations, and environmental disturbances are all introduced into the digital twin to add a level of robustness to the training process through the introduction of controlled perturbations.

ARTICLE IN PRESS

Equidistantly measured sensor values of the physical robot are mapped into the digital twin state space via the synchronization function detailed in Eq. (2), which aligns the physical sensors with their simulated counterparts at t.

$$x_{dt}(t) = \Phi(x_{pr}(t),\eta) \quad (2)$$

where $x_{dt}(t)$ is the digital twin state vector at time t, $x_{pr}(t)$ is the physical robot sensor state vector, and η denotes the parameters for calibration and compensation due to noise.

Eq. (2) provides temporal alignment between both the real-world robot and the digital version of it. The synchronization parameter vector η is initialized through offline calibration experiments using synchronized physical robot and digital twin trajectories. During operation, η is continuously refined using least-squares minimization between physical and simulated state trajectories to compensate for sensor drift and communication latency.

The digital twin serves as a source of high-fidelity data that provides structured and physical validated data to the learning module. In addition, the difference between the simulated trajectories and the actual physical trajectory is used to trigger training updates by the RPA layer.

Integration Flow between Digital Twin and Physics-Discovering Neural Model

The integration of the Digital Twin with the Physics-Discovering Neural Network (PDNN) is intended for continuous discovery of physics and adaptively controlling the system. The data generated by the digital twin are partitioned into state trajectories, control inputs, and disturbance profiles. These data streams then feed into the neural model, where it attempts to learn the state evolution and the governing physical laws that describe the dynamics of the system.

Let the neural model estimate the state of the system as a continuous function over time as described in Eq. (3).

$$\hat{x}(t) = f_\theta(t,u(t)) \quad (3)$$

where $\hat{x}(t)$ is the predicted state vector, f_θ is the neural network function whose weights are represented by θ, and $u(t)$represents the control input.

According to Eq. (3), the purpose of the neural architecture is to predict the states of the system and to also discover the implicit physical structure describing the system's dynamic evolution. The neural network will predict the states based on the outputs provided by the digital twin to generate residuals from that comparison. Additionally, the constraints of the dynamics are included in the loss function as given by Eq. (1) so that the neural network learns to satisfy the equations governing the robot's motion.

The constraint that the predicted states of the system always predict physical structure of the system will be determined by creating a residual given in Eq. (4).

$$R_{phys}(t) = M(\hat{q})\ddot{\hat{q}} + C\left(\hat{q},\dot{\hat{q}}\right)\dot{\hat{q}} + G(\hat{q}) + \hat{F_d} - \tau \quad (4)$$

where $\hat{q}$, $\dot{\hat{q}}$, and $\ddot{\hat{q}}$ indicate the predicted joint position, velocity, and acceleration generated from the neural model and $\hat{F_d}$represents the learned disturbance and friction term estimated by the PDNN, including variable friction coefficients, actuator degradation effects, and environmental disturbances.

The method by which Eq. (4) imposes the physical law constraint is by penalizing any deviations from the dynamics of the robot. By doing so, this approach allows the model to determine unknown (or at least partially) and/or unknown-to-be-determined quantities (e.g., friction coefficients, variations in inertia due to payload).

ARTICLE IN PRESS

The digital twin uses inputs from the neural model to continually update its internal parameters. The bidirectional nature of this relationship creates an environment in which the digital twin and neural model will continue developing together – co-adaptively. If the residual defined in Equation (4) exceeds a predetermined threshold, the RPA module initiates automated retraining and recalibration of the parameters.

Automated Control Signal Validation and Feedback Loop

As soon as the neural model produces predicted states and inferred dynamics, these results are validated within the digital twin prior to being implemented on the physical robot. The control policy produced from the neural network is expressed in Eq. (5).

$$u(t) = \pi_\theta(\hat{x}(t)) \tag{5}$$

where $u(t)$ is the control command and π_θ represents the control policy learned from the neural weights.

In Eq. (5), there is a direct mapping from predicted system states to control actions. These control signals are first evaluated via testing within the digital twin environment for stability, energy consumption, and trajectory accuracy. The performance evaluations produced from this testing are compared against predefined constraints, and only validated policies are ultimately sent to the physical robot.

Non-invasive feedback for control commands will prevent implementation of unsafe commands in the actual system. A self-improving feedback loop between the neural model and the digital twin provides ongoing enhancements to both the control performance and physical interpretability of the system implementation.

The integration flow between modules thus proceeds as follows: the digital twin generates synchronized state data using Eq. (2), the neural model predicts future states through Eq. (3), physics consistency is enforced via Eq. (4), and validated control actions are generated using Eq. (5). This closed-loop pipeline operates autonomously under the supervision of the RPA orchestration layer, which coordinates data acquisition, training, validation, and deployment.

Physics-Discovering Neural Network Core

The PDNN is the computational kernel of the SO-PDNA framework and expands upon standard PINNs by conducting adaptive discoveries of unknown physical relationships as opposed to restricting learning to a finite set of predetermined differential equations. The PDNN will produce a universal system state and latent dynamics via the synchronized digital twin data while preserving physics consistency.

For continuous time-based robot-state prediction, PDNN will create a deep neural network function mapping time/controls to joint states as defined in Eq. (6).

$$\hat{x}(t) = N_\theta(t,u(t)) \tag{6}$$

where $\hat{x}(t)$ represents the expected system state, N_θ represents the neural network parameterized by weights θ, and $u(t)$ denotes a vector of control input to influence the robot's state.

The underlying time evolution of how a robot moves through its environment can be captured by the neural model (outlined in Eq. 6). Unlike standard black box neural networks, the PDNN integrates physical constraints directly into its learning criteria, thereby guaranteeing that any predicted parameter be consistent to the actual dynamic behaviour of that robot.

ARTICLE IN PRESS

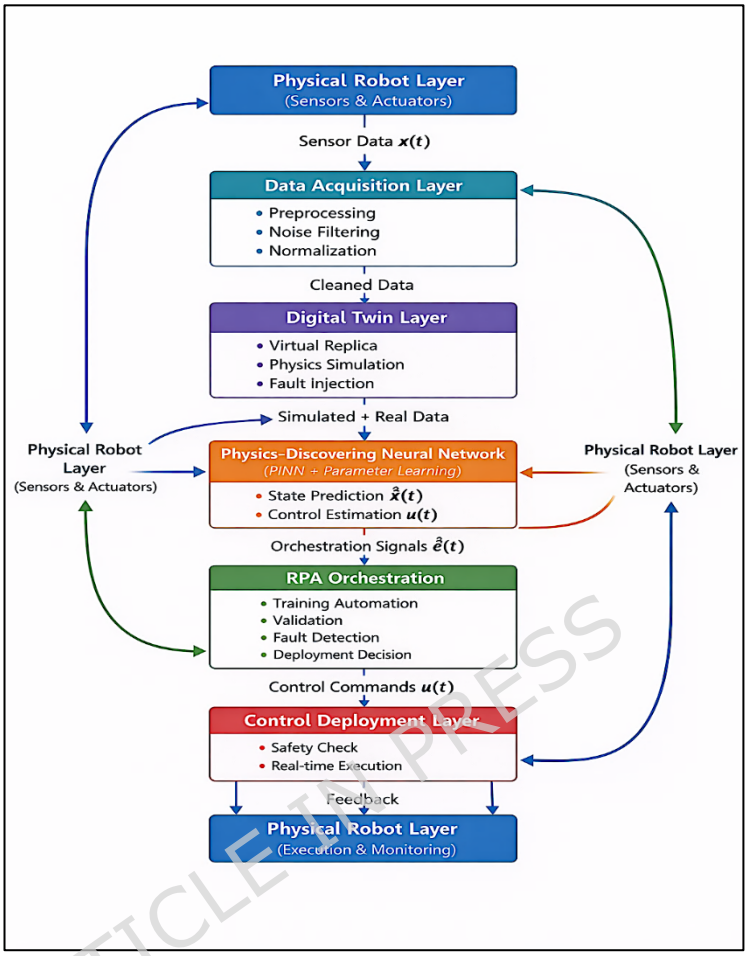


Figure 1. Architectural diagram of the proposed self-orchestrating robotic system

In addition, the PDNN also employs automatic differentiation to calculate time derivatives of the predicted state, thus allowing the PDNN to estimate what velocities and accelerations the robot would be subjected to, such that the required physics-based constraints can be enforced. The time derivatives of the predicted state (as computed in Eq. 7) are intended to ultimately be utilized for the integration of the governing physics-based equations during the neural optimization process.

$$\dot{\hat{x}}(t) = \frac{d}{dt}N_\theta(t,u(t)), \ddot{\hat{x}}(t) = \frac{d^2}{dt^2}N_\theta(t,u(t)) \qquad (7)$$

where $\dot{\hat{x}}(t)$ and $\ddot{\hat{x}}(t)$ denote the first and second derivatives of the predicted state vector with respect to time.

To evaluate the physical consistency of these predictions, the time derivatives of the predicted state as given in Eq. 7 will be substituted into the residual dynamics equation that is defined in Part I, Eq. 4.

Physics-Guided Loss Function Formulation

The objective of training the PDNN is to minimize the composite loss function, which consists of the sum of the differences between the actual and predicted data (data fitting error), the

ARTICLE IN PRESS

governing physical equation residuals (governing physics residual), and the smoothness constraints (control regularizing) that are required to maintain control of the robot. A multi-objective optimization strategy will ensure a balance between accuracy and physical feasibility in the resulting predictions. The complete loss function to be evaluated during training is defined in Eq. 8.

$$L_{total} = \lambda_d L_{data} + \lambda_p L_{phys} + \lambda_c L_{ctrl} \quad (8)$$

where L_{total} is the total loss of training, L_{data} is the data fitting loss, L_{phys} is the physics residual loss, L_{ctrl} is the control regularization loss, and λ_d, λ_p, and λ_c are weighting coefficients for the control regularization loss.

The data-driven component of the loss function has been derived using the difference between predicted states and the corresponding states from the digital twin (ground truth), expressed in Equation (9).

$$L_{data} = \frac{1}{N}\sum_{i=1}^{N} \| x_{dt}(t_i) - \hat{x}(t_i) \|^2 \quad (9)$$

where $x_{dt}(t_i)$ is the state of the digital twin at time t_i, $\hat{x}(t_i)$ is the corresponding predicted state, and N is the number of training samples.

As stated in Eq. (9), the aim of the neural network is to learn accurate representations of the observed behaviour of the robot by reproducing the digital twin generated robot trajectories.

The physics residual loss in terms of ensuring compliance with the robot dynamic model, is expressed in Eq. (10).

$$L_{phys} = \frac{1}{N}\sum_{i=1}^{N} \| R_{phys}(t_i) \|^2 \quad (10)$$

where $R_{phys}(t_i)$ is the calculated value of the physics residual as described in Eq. (4).

The use of Eq. (10) imposes penalties for violations of the physical laws controlling the motion of the robot and therefore supports the network in identifying unknown dynamic parameters such as friction coefficients and variations in inertia. The control regularization term is formulated in Eq. (11) to enforce a degree of smoothness and stability in control outputs of the robot.

$$L_{ctrl} = \frac{1}{N}\sum_{i=1}^{N} \| u(t_i) - u(t_{i-1}) \|^2 \quad (11)$$

where $u(t_i)$ is the control input at time step t_i.

The control regularisation term represented by Eq. (11) supports the generation of control inputs with little or no abrupt fluctuations therefore generating physically realisable commands for motion of the robot. Together, Eq. (8-11) combine the different components into a balanced training goal that equalises the fidelity of the data used to train the network against the physics-based consistency, and stability and therefore ensures the generation of control commands.

Physics Discovery and Latent Parameter Identification

The PDNN's unique characteristic is finding hidden physical parameters that are not given within the original dynamic model, including variable friction coefficients, actuator degradation effects, and external disturbance forces. These latent physical quantities are treated as trainable parameters within the neural architecture and are continuously updated

ARTICLE IN PRESS

during optimization. The extended dynamic representation of the dynamic model, including unknown parameters and disturbance-related dynamics, is defined in Eq. (12).

$$M(q;\alpha)\ddot{q} + C(q,\dot{q};\beta)\dot{q} + G(q;\gamma) + F_d(q,\dot{q};\delta) = \tau \quad (12)$$

where α, β, and γ are unknown parameter vectors associated with inertia, Coriolis effects, and gravitational forces respectively, while δ denotes the trainable latent parameters corresponding to frictional effects and external disturbances.

The introduction of unknown parametric uncertainty into the dynamic model is done through Eq. (12). These parameters, including the disturbance-related latent parameters δ, are concurrently learned together with the neural weights θ through backpropagation by minimizing the physics-based residual loss defined in Eq. (10). This enables the PDNN to continuously estimate unknown friction dynamics, environmental perturbations, and actuator degradation effects directly from synchronized robot and digital twin data. By doing this, the model can learn to find hidden physical relationships through data instead of pre-formulated mechanical equations.

A sparsity-enforcing constraint for discovering physical relationships from data is also included as part of the discovery method with regard to obtaining the dominant physical terms throughout the process. This is defined in Eq. (13).

$$L_{sparse} = \| \omega \|_1 \quad (13)$$

where ω is the coefficient vector associated with the candidate physical basis functions.

The implementation of Eq. (13) enforces sparsity within the learned representation such that interpretable physical laws can be found within the high-dimensional neural embeddings resulting from the learning process. Thus, the sparsity loss is added to the total loss formulation by amending Eq. (8) to include an additional term.

Integration Flow between Physics Discovery and Control Policy Learning

The dual-feedback mechanism facilitates the flow of information between the physics discovery and control policy learning processes. The initial flow occurs by injecting the discovered parameter values in Eq. (12) into the control policy formulation of Eq. (14), which allows for adaptive control when changes occur in-system conditions. The second flow occurs by providing performance metrics of the control system back to the physics discovery process to improve the estimates of the discovered parameter values.

The adaptive control policy with discover dynamics is presented by Eq. (14).

$$u(t) = \pi_\theta\left(x(t),\hat{\alpha},\hat{\beta},\hat{\gamma},\hat{\delta}\right) \quad (14)$$

where $\hat{\alpha}$, $\hat{\beta}$, $\hat{\gamma}$, and $\hat{\delta}$ denote the learned estimates of the unknown physical parameters, including latent friction and disturbance dynamics. The relationship represented by Eq. (14) shows that control actions can be developed based on more than just system states; these actions can also include physical properties of the system that are learned through a dynamic process. This connection between the controller's ability to react to changes in things like the payload, friction, and outside disturbances.

The digital twin validates the control policy it learns by simulating how it performs under several different perturbation scenarios. Deviations of expected performance versus the actual performance are measured and used to update both the robotic PDNN and the physical properties that have been estimated or discovered. Through this closed-loop integration, we ensure that we move towards stable and physically understandable control methods.

ARTICLE IN PRESS

RPA-Based Self-Orchestration Engine

The self-orchestration engine serves as a custom-built supervisory orchestration layer in the SOPDNA framework. Unlike conventional enterprise RPA software designed for UI-level business automation, the proposed orchestration engine operates as a lightweight finite-state supervisory controller responsible for coordinating anomaly monitoring, retraining triggers, digital twin synchronization, validation, and adaptive policy deployment. The primary function of this self-orchestrating engine is to automate the entire learning and control cycle, from ingesting data from the digital twin; to training and validating the physics-discovered neural model; to implementing the validated control policies on the actual robot. Unlike current robotic control pathways that rely on user intervention for scheduling and parameter tuning, the self-orchestration engine will monitor performance of the system continuously and will automatically take corrective actions when an anomaly occurs. The orchestration layer does not participate directly in the deterministic low-level motor control loop operating at the 100 Hz sampling frequency. Instead, real-time trajectory tracking and actuator control are handled independently by the PDNN controller, while the orchestration engine supervises higher-level asynchronous learning and adaptation tasks. This architectural separation minimizes latency overhead and prevents communication bottlenecks commonly associated with traditional commercial RPA platforms.

The RPA self-orchestration engine functions as a finite state orchestration process and can transition from one training state, validating state and deploying state according to the rules that we have developed. The orchestration rules can be mathematically represented using a state transition function defined in Eq. (15).

$$S_{t+1} = \Psi(S_t, \epsilon_t) \tag{15}$$

where S_t is the current orchestration state, S_{t+1} denotes the next state, and ϵ_t corresponds to the observed system error metric at time t.

Autonomous switching between supervisory operating modes is accomplished with Eq. (15), where the orchestration engine functions as a finite-state coordination mechanism for adaptive retraining, validation, and deployment tasks rather than a low-level real-time controller. Any time the observed error is greater than a threshold, the RPA engine switches the system into retraining mode. On the other hand, if the performance is within acceptable bounds, the RPA engine allows the system to remain in deployment mode.

The error metric that drives orchestration decisions can be computed using the performance deviation function shown in Eq. (16).

$$\epsilon_t = \| x_{pr}(t) - \hat{x}(t) \| \tag{16}$$

where $x_{pr}(t)$ denotes the actual robot state and $\hat{x}(t)$ denotes the predicted robot state selected using the physics-discovering neural model.

Equation (16) quantifies the difference between the actual robot performance and the learned dynamics. The RPA engine calculates this metric continuously to determine if retraining/recalibration is needed based on the continuous evaluation of whether the sampled performance is above the preset threshold of acceptable performance.

Automated Fault Detection and Recovery Module

The RPA engine has also incorporated fault detection and recovery functionality to ensure safe and effective robot operation. On occasion, faults can be introduced to the system due to sensor noise, actuator degradation, or unexpected disturbances in the environment. The fault indicator is defined as the statistical anomaly score that can be determined using Eq. (17).

ARTICLE IN PRESS

$$F(t) = \frac{\epsilon_t - \mu_\epsilon}{\sigma_\epsilon} \quad (17)$$

where $F(t)$ is the fault indicator, μ_ϵ denotes the historical mean error, and σ_ϵ is the standard deviation of the error.

Eq. (17) provides a measure of the normalized deviation from the expected level of performance. If the fault indicator exceeds the set critical threshold, an automated recovery sequence is initiated. This will include isolating any corrupted data, invoking digital twin re-simulation, and retraining of the PDNN with the new parameterized values for the operating system.

The recovery mechanism further adapts the control policy to compensate for detected faults. The modified control command is defined in Eq. (18).

$$u_f(t) = u(t) - K_f F(t) \quad (18)$$

where $u_f(t)$ denotes the fault-compensated control input, $u(t)$ is the nominal control command from Eq. (14), and K_f represents the fault compensation gain.

Equation (18) ensures that control actions remain stable even under abnormal operating conditions. The fault compensation gain K_f is experimentally initialized using closed-loop stability analysis and subsequently fine-tuned through adaptive gain scheduling based on the magnitude of the detected fault indicator F(t). This adaptive adjustment ensures stable recovery behavior under varying actuator degradation conditions. This adaptive fault mitigation strategy contributes to the self-healing capability of the SOPDNA framework.

Algorithmic Workflow

The complete operation of the proposed system is governed by an integrated algorithm that coordinates digital twin simulation, physics discovery, control policy learning, and RPA orchestration. The overall learning and control process is described through a recursive update formulation expressed in Eq. (19).

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta L_{total} \quad (19)$$

where θ_k and θ_{k+1} denote the neural parameters at iteration k and $k + 1$, η is the learning rate, and L_{total} is the composite loss defined in Eq. (8).

Equation (19) governs the optimization of the physics-discovering neural model. Each iteration updates both neural weights and latent physical parameters using feedback from the digital twin and physical robot.

The orchestration loop further integrates control policy deployment and validation. The convergence of the system is assessed using a stability criterion defined in Eq. (20).

$$\lim_{t \to \infty} \epsilon_t = 0 \quad (20)$$

where ϵ_t is the performance error defined in Eq. (16).

Equation (20) indicates that the system converges when the predicted and physical robot states align over time. The RPA engine monitors this convergence condition and maintains the system in deployment mode once stability is achieved.

ARTICLE IN PRESS

Deployment Pipeline and Continuous Learning Mechanism

The optimized control policy is sent to the physical robot through a secure communication interface once validated in the digital twin environment. This conduit will allow the deployed controller to adapt to emerging sensor observations and to adjust the learned values of physical parameters.

The deployed policy update rule is formulated in Eq. (21).

$$\pi_{t+1} = \pi_t + \Delta\pi\left(\theta, \hat{\alpha}, \hat{\beta}, \hat{\gamma}\right) \tag{21}$$

where π_t is the control policy before its update and π_{t+1} is the control policy after its update, and $\Delta\pi$ is the policy refinement function that adjusts for all the learned constants used to refine its value. The policy refinement function $\Delta\pi$ is determined using gradient-based incremental policy optimization derived from the composite loss function defined in Eq. (8). The refinement process incorporates both the updated neural parameters and the discovered physical parameters to ensure stable continuous adaptation without abrupt policy transitions.

Eq. (21) ensures that the controller evolves incrementally rather than through abrupt replacements, preserving stability and continuity of robot motion.

Through continuous learning, the engineered system can adjust to any new operating condition; for example, with respect to the presence of payload or the introduction of disturbances into the environment. The digital twin can effectively mimic the newly emerging conditions and will also provide newly generated training data to the PDNN, thereby enabling it to further refine its acquired physical laws and control methods.

System-Level Integration and Convergence

The SOPDNA framework brings together Digital Twin simulations, Physics-Discovering Neural Networks, and RPA orchestrated autonomous control systems to create a single, unified autonomous control architecture. The integration flow starts from digital twin creation, to physics discovery via Eqs. (6)-(13), to adapting control policies via Eq. (14), and automatically orchestrating via Eqs. (15)-(21). This end-to-end pipeline allows for self-learning, self-validating, and self-healing all within one cohesive system. The entire framework converges through the joint minimization of data error, physics residuals, and control instability. The convergence of the system is described via Eq. (20) - where the robot's physical behavior has been shown to converge to a stable equilibrium with its predicted dynamics. This convergence property provides long-term reliability and robustness for real-world robotic applications. As such, the proposed SOPNA framework represents a shift in paradigm from static robots to self-orchestrating intelligent robot systems that will discover the physical laws impacting their operation, adapt to dynamic environments, and autonomously manage their entire learning lifecycle. By bringing together digital twins with physics-discovering neural networks and RPA based orchestration, the SOPNA framework establishes a flexible, interpretable, and reproducible basis for next-generation autonomous robot motion control.

Algorithm 1: SOPDNA for Autonomous Robotic Motion Control

Input:
Physical robot sensory data $x_{pr}(t)$, initial Digital Twin model DT_0, neural network parameters θ_0, learning rate η, error

ARTICLE IN PRESS

threshold ε_{th}, fault threshold F_{th}, maximum iterations K

Output:
Optimized control policy π^*, discovered physical parameters $\hat{\alpha},\hat{\beta},\hat{\gamma}$

1: Initialize neural parameters $\theta \leftarrow \theta_0$
2: Initialize Digital Twin state $DT \leftarrow DT_0$
3: Initialize orchestration state S_0
← "Training"

4: for $k = 1$ to K do
5: Acquire real-time sensory data from physical robot: $x_{pr}(t_k)$

6: Synchronize Digital Twin with physical robot state:
7: $x_{dt}(t_k) \leftarrow \Phi(x_{pr}(t_k))$

8: Generate simulated trajectories and disturbances using Digital Twin:
9: $\{x_{dt}(t_i),u(t_i)\}_{i=1}^{N} \leftarrow DT(x_{dt}(t_k))$

10: Predict robot system states using Physics-Discovering Neural Network:
11: $\hat{x}(t_i) \leftarrow N_\theta(t_i,u(t_i))$

12: Compute physics residual based on robot dynamics:
13: $R_{phys}(t_i) \leftarrow M(\hat{q})\ddot{\hat{q}} + C(\hat{q},\dot{\hat{q}})\dot{\hat{q}} + G(\hat{q}) - \tau$

14: Evaluate total loss function:
15: $L_{total} \leftarrow \lambda_d L_{data} + \lambda_p L_{phys} + \lambda_c L_{ctrl}$

16: Update neural parameters and latent physical parameters:
17: $\theta_{k+1} \leftarrow \theta_k - \eta \nabla_\theta L_{total}$
18: $(\hat{\alpha},\hat{\beta},\hat{\gamma}) \leftarrow$ Optimize via backpropagation

19: Generate adaptive control policy:
20: $u(t) \leftarrow \pi_\theta(x(t),\hat{\alpha},\hat{\beta},\hat{\gamma})$

21: Validate control policy in Digital Twin environment:
22: Simulate robot motion using $u(t)$

ARTICLE IN PRESS

ARTICLE IN PRESS

23: Compute performance error:
24: $\varepsilon_t \leftarrow \| x_{dt}(t) - \hat{x}(t) \|$

25: RPA-based orchestration decision:
26: if $\varepsilon_t > \varepsilon_{th}$ then
27: Set $S_{k+1} \leftarrow$ "Retraining"
28: Trigger Digital Twin recalibration and model retraining
29: else
30: Set $S_{k+1} \leftarrow$ "Deployment"
31: Deploy control policy to physical robot
32: end if

33: Fault detection and recovery mechanism:
34: Compute fault indicator:
35: $F(t) \leftarrow (\varepsilon_t - \mu_\varepsilon)/\sigma_\varepsilon$

36: if $F(t) > F_{th}$ then
37: Apply fault-compensated control:
38: $u_f(t) \leftarrow u(t) - K_f F(t)$
39: end if

40: Update Digital Twin state and orchestration status:
41: $DT \leftarrow DT + \Delta DT$
42: $S_k \leftarrow S_{k+1}$

43: end for
44: Return optimized control policy π^* and discovered physical parameters $(\hat{\alpha}, \hat{\beta}, \hat{\gamma})$

ARTICLE IN PRESS

IV. RESULTS AND DISCUSSION

In this section, we will evaluate the proposed SOPDNA for autonomous robotic motion control. The evaluation was done via experiments, performed under dynamic and uncertain conditions, and compared with other traditional controllers and with other learning-based methods. The results of these experiments demonstrate the effectiveness of the SOPDNA framework in tracking accuracy, energy use efficiency, robustness to disturbances, and recovering from faults.

Robotic Platform and Digital Twin Environment

The SOPDNA framework was experimentally evaluated using a Universal Robots UR5e six-degree-of-freedom (6-DOF) robotic manipulator in both physical and digital twin environments. The UR5e robotic platform features six revolute joints, a maximum payload capacity of 5 kg, and a repeatability of ±0.03 mm. The robot was interfaced with a high-performance workstation equipped with an Intel Core i9 processor, NVIDIA RTX 4090 GPU, and 64 GB RAM to support real-time neural training, digital twin synchronization, and adaptive control deployment. The physical robotic manipulator was equipped with integrated joint encoders, force–torque sensors, inertial measurement units (IMU), and proximity

ARTICLE IN PRESS

sensors to acquire real-time motion and environmental interaction data at a sampling frequency of 100 Hz.

Dataset Description and Statistics

The SOPDNA training and evaluation data set was created using both high-fidelity digital twin simulations and execution of the robotic manipulator. The resultant data set contains motion trajectories of the robotic manipulator for a range of operational conditions that include differences in payload, different coefficients of friction, and external disturbances being applied to the robotic manipulator. The data set was created to expose the SOPDNA learning framework to nominal and perturbed dynamic behaviour to achieve a robust generalisation to previously unseen scenarios. Each data sample in the created data set is represented as in Eq. 22.

$$D = \{q(t), \dot{q}(t), \ddot{q}(t), \tau(t), F_{ext}(t)\} \quad (22)$$

The joint position is denoted as $q(t)$ while the joint velocity is indicated as $\dot{q}(t)$, the joint acceleration is indicated as $\ddot{q}(t)$, the actuator torque is indicated as $\tau(t)$, and the external disturbance force acting upon the robot is represented as $F_{ext}(t)$. The kinematic and dynamic states of the robotic system are reflected within these variables and provides sufficient information to determine the physical relationships in addition to learning the adaptive control policy. A summary of the attributes of the dataset are included in Table 2. Referring to Table 2, 120,000 samples were collected over 3,500 motion trajectories, at a sampling rate of 100Hz, of a six degrees of freedom robotic manipulator. The dataset encompasses variations in payload from 0.5 kg to 3.0 kg. The dataset also contains examples of different types of disturbances, including frictional changes, impulse forces, and sensor noise. Finally, 25 distinct obstacle interaction situations were created that test the adaptive control policy of the controller in a constrained environment. In total, 80% of the dataset was allocated for training and 20% was allotted for testing to create a conclusive performance evaluation for all evaluation conditions.

A digital twin (i.e., a virtual model) was developed as a high fidelity, digital replica of the robotic system that mimicked the kinematic and dynamic behaviour of the actual robot. The digital twin continually brought itself into a state of synchrony to real-time sensor feedback from the physical robot and therefore allowed large-scale experimentation (e.g., due to the time it takes to physically conduct an experiment) and provided a method for safely validating the adaptive control policy of the digital twin through experimentation. The digital twin also allowed for simulating a range of payload (e.g., payloads of varying weights), changing friction coefficients, and different types of external disturbances prior to applying control policies to the physical robot.

Table 2. Dataset Characteristics

Parameter	Value
Total samples	120,000
Number of trajectories	3,500
Degrees of freedom	6
Sampling rate	100 Hz
Payload range	0.5–3.0 kg
Disturbance types	Friction, impulse, noise
Obstacle scenarios	25
Training/Test split	80% / 20%

Performance Metrics

The proposed control framework was evaluated comprehensively for its effectiveness and robustness using multiple quantitative performance metrics that captured both control accuracy and system resilience under dynamic operating conditions. The accuracy of trajectory tracking was used as a key metric for assessing how well the actual motion of the

ARTICLE IN PRESS

robot matches the desired reference trajectories. This was represented as a percentage of the completion of the trajectory to enable comparisons of the performance of alternative controllers. In addition, Root Mean Square Error (RMSE) was calculated to provide an average measure of deviation between the predicted joint states and actual joint states over time, providing a more sensitive measure of control accuracy.

Energy consumption was included as one of the metrics for evaluating system efficiency by indicating the total energy required (in joules) to complete a defined task and reflecting the smoothness and physical consistency of the generated control signals. Settling time and overshoot were used as additional metrics for assessing temporal response characteristics of the control system and demonstrating how quickly the system reached steady-state behaviour after being subject to disturbances and how much temporary deviations from desired trajectory occurred. The metrics used are critical for safety-critical robotics systems where excessive oscillations or long time to convergence will affect the reliability of the system.

To determine the resilience of the controller with respect to disturbances originating outside the system, the disturbance (or disturbance to the output) rejection ratio indicated how effectively the controller can mitigate the effect of sudden (impulsive) and continuous disturbances on system performance. Lastly, the recovery time from a fault was used as an indicator of the adaptability of the control architecture in response to degradation of the actuator or anomalies in the sensor and measured the amount of time required to recover the system to stable operation following the detection of a fault. These five metrics together give a comprehensive evaluation of the accuracy, stability, energy efficiency, robustness, and autonomous recovery of the proposed framework across different experimental conditions.

Trajectory Tracking Performance

The proposed SOPDNA controller's trajectory tracking performance was assessed across nominal, variable payload, and external disturbance environments. Qualitative assessments of desired versus actual trajectories can be seen in Figure 2, while quantitative values can be found in Table 3, during nominal, variable payload, and external disturbance testing. The SOPDNA control system tracked the reference trajectory extremely closely with nearly no deviation, even when encountering external disturbance, whereas both PID and DRL did so with significant oscillations and steady-state error as shown in Figure 2. As shown in Table 3, the SOPDNA achieved the best overall tracking accuracy across all three conditions with 98.4% nominally, 96.2% with a variable payload, and 94.8% in disturbed conditions. Overall, this represents a 3.3% improvement in tracking accuracy from PID to SOPDNA in model-based disturbance scenarios and a greater than 9% improvement in accuracy relative to DRL. The performance improvements that SOPDNA achieved can primarily be attributed to three factors: (1) the physics-discovering neural core of the SOPDNA control system that continually estimates and refines the underlying latent parameters of the robotic system (e.g., friction and inertia based on a variable payload); (2) the digital twin module provides a method of using safe self-learning to refine the control policy; and (3) the novel design of SOPDNA provides a high degree of robustness and adaptability with respect to accurate robotic trajectory control.

Table 3. Tracking Accuracy (%)

Controller	Nominal	Variable Payload	Disturbance
PID	89.3	82.1	78.6
DRL	93.5	88.7	85.2
PINN	95.1	91.4	89.6
SOPDNA	**98.4**	**96.2**	**94.8**

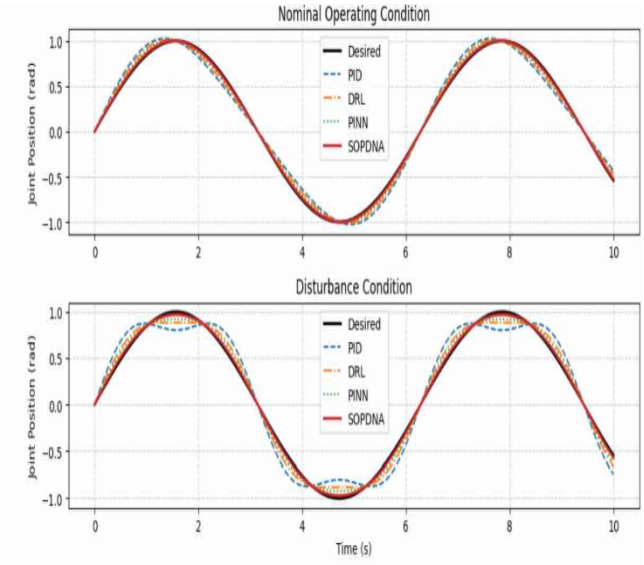


Figure 2. Desired and actual joint trajectories under nominal and disturbed operating conditions for different controllers.

Energy Efficiency Analysis

The experimental results presented in Table 4 and Figure 3, show that the proposed SOPDNA control system has the lowest energy consumption compared to other similar systems evaluated in this study. For example, the SOPDNA controller has demonstrated a 22.7% decrease in power consumption when compared to the traditional PID controller, thereby exhibiting improved efficiency. Both DRL and PINN also demonstrate a reduction in energy consumption, but do so at a lower level than SOPDNA due to their unrestrained control actions; thus, their performance is substantially inferior to SOPDNA. In addition to providing physically valid torque profiles, SOPDNA has the capability to prevent excessive, aggressive actuation by learning its own inherent system dynamics and integrating this information into its neural network architecture. Similarly, the validation and optimization of the SOPDNA controller's control laws can be performed safely within the digital twin environment prior to their deployment in the real-world environment. Therefore, by utilizing both physics-discovering learning and virtual optimization, SOPDNA provides the smoothest actuation possible and the least variation in torque while maintaining energy savings through all payload and disturbance scenarios, thus demonstrating that SOPDNA is well-suited for long-term autonomous robotic operation.

Table 4. Energy Consumption (J)

Controller	Energy (J)	Reduction vs PID
PID	100.0	–
DRL	87.6	12.4%
PINN	82.3	17.7%
SOPDNA	77.3	22.7%

ARTICLE IN PRESS

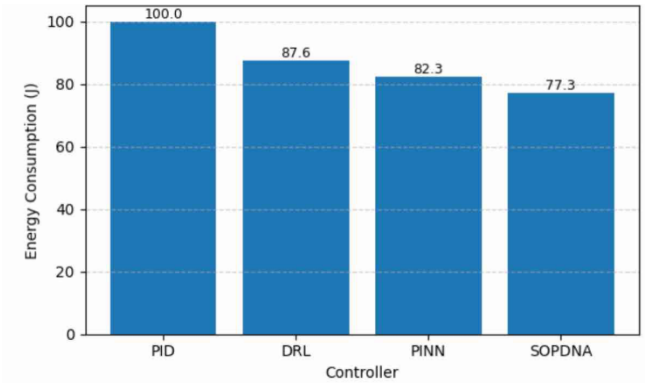


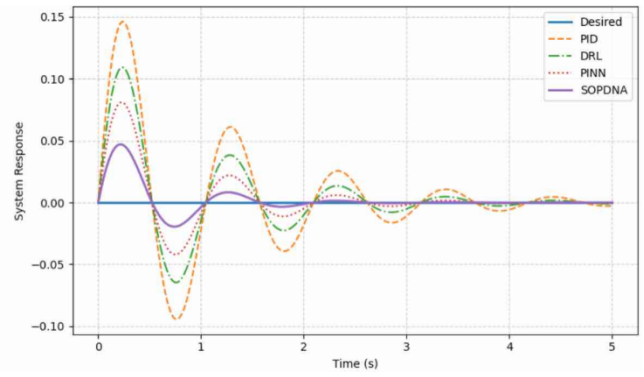
Figure 3. Energy consumption per control cycle for different controllers.

Disturbance Rejection Performance

The disturbance-rejection capability of the proposed SOPDNA controller was analyzed by applying an impulse-type external disturbance during the movement of each trajectory. The responses from each controller tested while under disturbance conditions are shown in Figure 4 and the quantitative disturbance rejection performance metrics from these tests are displayed in Table 5. The SOPDNA controller was able to produce responses that are very smooth with quick stabilization after an impulse disturbance is applied, in comparison to the other two controllers which demonstrated significant oscillations and deviations from their desired trajectory. The data presented in Table 5 collectively support this conclusion; the SOPDNA control had a minimal overshoot value of 7.5% and the quickest settling time of 1.4s while using PID control yielded an overshoot value of 18.4% with a settling time of 2.9s. The superior performance under disturbance conditions is due to having the orchestration mechanism based on RPA that enables continuous monitoring of performance degradation and the ability to automatically trigger model retraining when an external disturbance causes a change in the system dynamics. In addition, the physics-discovery neural core of the controller allowed for the development of a learnt internal representation of the latent physical parameters allowing for rapid compensation of unexpected perturbations and maintenance of stable operation in a dynamic environment.

Table 5. Disturbance Rejection Metrics

Controller	Overshoot (%)	Settling Time (s)
PID	18.4	2.9
DRL	14.2	2.4
PINN	11.7	1.9
SOPDNA	**7.5**	**1.4**



ARTICLE IN PRESS

Figure 4. System Response under Impulse Disturbance

Fault Detection and Recovery Analysis

An actuator degradation fault equivalent to losing 20% of the actuator's efficiency was simulated to evaluate the fault detection and recovery performance. This study generated temporal recovery performance illustrations shown in Figure 5 and recovery times in Table 6. As shown in Figure 5, the PID and DRL controllers' deviations from the reference trajectory were prolonged after the fault was injected, indicating that these controllers adapted more slowly to abnormal operating conditions than SOPDNA. However, SOPDNA quickly restored stable tracking operations after the fault occurred. Quantitatively, Table 6 demonstrates that SOPDNA achieved a 3.6-second recovery time, compared to the 6.14 second recovery time for PID control (41.3%). The principal reason for this improved performance is the orchestration layer in SOPDNA that autonomously detects abnormal system behaviour and triggers recalibration of the model without requiring human intervention. The digital twin also allowed for safely retraining and validating the updated control policies prior to deployment, therefore ensuring reliable and rapid fault recovery for real-world robots.

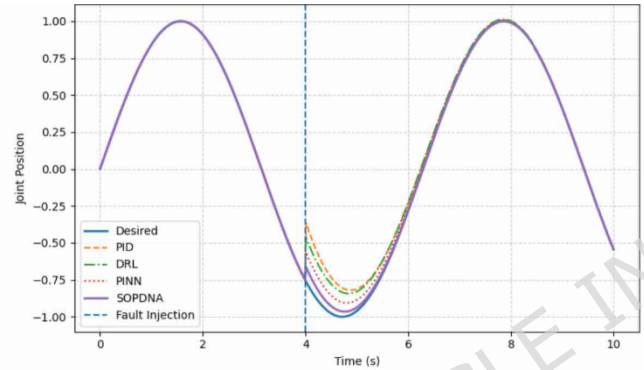


Figure 5. Fault recovery under actuator degradation.

Table 6. Fault Recovery Time

Controller	Recovery Time (s)
PID	6.2
DRL	5.1
PINN	4.4
SOPDNA	**3.6**

Generalization to Unseen Payloads

To evaluate the capability of generalising decisions made by the controllers, analyses were carried out using payloads not represented in the learning set (payload). The root mean square errors (RMSEs) calculated for the three different payload cases are presented in Table 7. It can be seen from the RMSE values that SOPDNA displays the lowest levels of RMSE for all payload conditions with values being 0.018, 0.025 and 0.039 for payloads of 0.5 kg, 1.5 kg and 3.0 kg, respectively. In contrast, both the PID and DRL controller errors exhibit increasing errors with increasing payloads and demonstrate a lack of extrapolation capability outside their training range. The superior performance of SOPDNA is attributed to its physics discovery mechanism, where the controller learns and understands underlying dynamic relationships such as variances in inertia and friction, as opposed to developing purely data-driven correlations, allowing the controller to generalise to new and untested payloads and

ARTICLE IN PRESS

confirming its robustness and scalability to real world robotic applications with varied physical loads.

Table 7. RMSE under Payload Variations

Payload (kg)	PID	DRL	PINN	SOPDNA
0.5	0.042	0.031	0.026	**0.018**
1.5	0.067	0.052	0.041	**0.025**
3.0	0.093	0.078	0.063	**0.039**

Ablation Study

The purpose of running an ablation study was to assess the role of specific elements in the SOPDNA architecture. The qualitative analysis of alternative designs can be seen in Figure 6 and the overall results are shown in Table 8. The complete configuration of SOPDNA achieved a total accuracy rate of 98.4% with an average recovery time of 3.6 seconds. By removing the Digital Twin Module from the architecture, the accuracy rate decreased to 95.2% and the recovery time increased to 4.8 seconds. This demonstrates that the use of virtual safe validation methods is essential prior to the deployment of the system. The removal of the RPA Orchestration Component did, however, further hinder the overall performance of the entire system, as this component was responsible for providing independent adaptation and management of the overall lifecycle of the system. The removal of all physics constraints was the most damaging single element, resulting in an accuracy rate of 92.1% and a recovery time of 6.1 seconds. These results clearly show that each component of the proposed architecture is critical to supporting the stability, interpretability, and reliability of the overall solution.

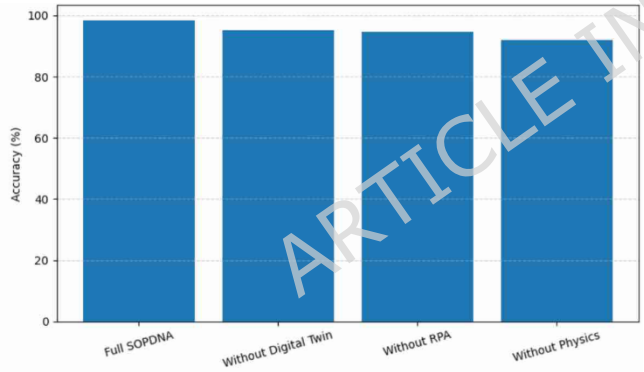


Figure 6. Contribution of SOPDNA components.

Table 8. Ablation Results

Configuration	Accuracy (%)	Recovery Time (s)
Full SOPDNA	**98.4**	**3.6**
Without Digital Twin	95.2	4.8
Without RPA	94.6	5.2
Without Physics Constraints	92.1	6.1

Statistical Significance Analysis

A statistical comparison between different controllers was undertaken to determine performance consistency and dependability over multiple trials. The findings of this study can be found in Table 9. SOPDNA demonstrates the highest overall average tracking accuracy at 98.1% ± 0.7, while also having the least variance among all controllers. In regard

ARTICLE IN PRESS

to energy usage, SOPDNA utilizes 77±2.9 J, compared with PID, DRL and PINN, whose energy consumption values were significantly higher. The lesser standard deviation for SOPDNA also indicates that SOPDNA exhibits higher performance capabilities, as well as greater levels of stability and repeatability between testing iterations. These findings provide evidence that the improvements noted in the data are due to the application of physics-based learning and orchestration mechanisms and not coincidental.

Table 9. Statistical Comparison (Mean ± SD)

Metric	PID	DRL	PINN	SOPDNA
Accuracy (%)	88.4±2.1	92.3±1.8	94.2±1.3	**98.1±0.7**
Energy (J)	100±5.4	87±4.1	82±3.6	**77±2.9**

Overall Discussion

The experimental evidence provided indicates that the proposed SOPDNA framework is able to integrate the discovery of physics, the validation of digital twin technology, and the orchestration of RPA properties into one customizable and adaptive control system. The SOPDNA framework is not like traditional control systems, which have fixed models or offline-trained neural networks. Instead, the SOPDNA framework continually updates its internal representation of the dynamic physical world and independently manages its own learning processes. The results demonstrated in Figures 2–6 and Tables 3-9 support the claim that SOPDNA outperforms the ability of traditional control systems to reject disturbances, recover from faults, generalize to previously unseen payloads, and achieve statistically significant improvements in accuracy or energy efficiency. In addition, the ablation study validated the necessity of each architectural component included in the SOPDNA framework for achieving these improvements. Thus, the SOPDNA framework is an intelligent and robust control paradigm that minimizes the requirement for human oversight and maximizes the reliability and adaptability of complex robotic platforms.

V. CONCLUSIONS AND FUTURE WORK

This paper describes a SOPDNA for controlling an autonomous robot's motion intelligently and effectively by using a combination of physics-informed neural networks, RPA, and digital twin technologies. Traditional approaches to controlling a robot's motion use either an analytical or mathematical model that has been predetermined or some other existing model, such as human labor used to create a control system and learning pipeline. The SOPDNA approach enables an autonomous robot using this technology to automatically discover new or unknown dynamics within its physical environment. Once these dynamics are identified, the SOPDNA architecture can orchestrate the data learning processes based on the discovered dynamics and validate its methods of control through continuous interaction with the digital twin. The RPA component provides the added benefits of automating the acquisition of data, performing anomaly detection, retraining on an adaptive basis, and deploying optimized control policies, which results in a completely automated learning and control cycle for the robot.

The experimental results demonstrate that the proposed SOPDNA framework achieves improved trajectory tracking accuracy, disturbance rejection capability, energy efficiency, and fault recovery performance compared with PID-, DRL-, and PINN-based baseline approaches under the evaluated experimental conditions. In addition, the fault injection experiments indicate that the integration of physics-discovering neural learning, digital twin validation, and automated orchestration can improve adaptive control behavior under actuator degradation and external disturbances. These findings indicate the potential of physics-discovering neural-based architectures for adaptive robotic motion control under

ARTICLE IN PRESS

controlled experimental conditions, while broader validation across external benchmark datasets, heterogeneous robotic platforms, and real-world deployment scenarios remains an important direction for future research.

Despite the encouraging experimental results, the present study has several limitations. The evaluation was conducted using a self-generated dataset derived from a single robotic platform and digital twin environment, without validation on external benchmark datasets or heterogeneous robotic systems. In addition, the experiments were performed under controlled laboratory conditions, which may not fully capture the complexity and variability of large-scale real-world deployments. Therefore, further investigation is required to assess the scalability, computational efficiency, and generalization capability of the proposed framework across diverse robotic applications and operating environments.

The framework's future development will be aimed at allowing cooperative discovery of physical laws through multi-robot and swarm robotic technology and at having these systems coordinate their movement with each other. Symbolic regression and advanced transformer-based neural networks will be the primary sources of research information for improving how well the discovered physical laws can be interpreted and generalized by the user. The framework will also be developed for use on embedded robotic hardware, as this will make the framework more usable due to meeting time constraints and increasing computational efficiency. Additionally, the development will include human-in-the-loop systems that provide feedback to the robotic system, as well as multi-modal sensor inputs that include vision and touch, so that robots can work collaboratively with humans. Finally, large-scale validation of the practical applicability and reliability of the robotic framework will take place in three areas: industrial automation, rehabilitation robotics, and autonomous navigation.

Declarations

Funding: The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests: The authors have no relevant financial or non-financial interests to disclose.

Data Availability: Data Availability: The raw data supporting the conclusions of this article will be made available by the authors, without undue reservation.

Consent for Publication: All authors have read and approved the manuscript.

Author Contributions:

Conceptualization, G. Logeswari; methodology, Daksh Singla, G. Logeswari, K. Tamilarasi, and J. Deepika Roselind; formal analysis, G. Logeswari and J. Deepika Roselind; investigation, G. Logeswari, Daksh Singla, and K. Tamilarasi; writing—original draft preparation, Daksh Singla, G. Logeswari, K. Tamilarasi, and J. Deepika Roselind; writing—review and editing, Daksh Singla, G. Logeswari, and K. Tamilarasi; visualization, Daksh Singla and G. Logeswari; supervision, G. Logeswari and K. Tamilarasi. All authors have reviewed the manuscript.

REFERENCES

[1] Sivtsov, V., Papanikolaou, A., Markovic, I., Petrovic, I., & Bonsignorio, F. (2025). Physics-informed neural networks in robotics: A review. Available at SSRN 5125543.

[2] Deng, W., Ardiani, F., Nguyen, K. T., Benoussaad, M., & Medjaher, K. (2024). Physics informed machine learning model for inverse dynamics in robotic manipulators. Applied Soft Computing, 163, 111877.

[3] Edwards, G., Garcia, J., Tajalli, H., Popescu, D., Medvidovic, N., Sukhatme, G., & Petrus, B. (2009, May). Architecture-driven self-adaptation and self-management in robotics systems. In 2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems (pp. 142-151). IEEE.

[4] Yendluri, D. K., Tatikonda, R., Thatikonda, R., Ponnala, J., Kempanna, M., & Bhuvanesh, A. (2023, December). Integration of SAP and Intelligent Robotic Process Automation. In 2023 International Conference on Next Generation Electronics (NEleX) (pp. 1-6). IEEE.

ARTICLE IN PRESS

[5] Qian, C., Liu, X., Ripley, C., Qian, M., Liang, F., & Yu, W. (2022). Digital twin—Cyber replica of physical things: Architecture, applications and future research directions. Future Internet, 14(2), 64.
[6] Mihai, S., Yaqoob, M., Hung, D. V., Davis, W., Towakel, P., Raza, M., ... & Nguyen, H. X. (2022). Digital twins: A survey on enabling technologies, challenges, trends and future prospects. IEEE Communications Surveys & Tutorials, 24(4), 2255-2291.
[7] Mazumder, A., Sahed, M. F., Tasneem, Z., Das, P., Badal, F. R., Ali, M. F., ... & Islam, M. R. (2023). Towards next generation digital twin in robotics: Trends, scopes, challenges, and future. Heliyon, 9(2).
[8] Kazemi Naeini, H., Shomali, R., Pishahang, A., Hasanzadeh, H., Asadi, S., & Gholizadeh Lonbar, A. (2025). Pinn-dt: Optimizing energy consumption in smart building using hybrid physics-informed neural networks and digital twin framework with blockchain security. Sensors, 25(19), 6242.
[9] Mengaldo, G., Renda, F., Brunton, S. L., Bächer, M., Calisti, M., Duriez, C., ... & Laschi, C. (2022). A concise guide to modelling the physics of embodied intelligence in soft robotics. Nature Reviews Physics, 4(9), 595-610.
[10] Talamadupula, K., Benton, J., Kambhampati, S., Schermerhorn, P., & Scheutz, M. (2010). Planning for human-robot teaming in open worlds. ACM Transactions on Intelligent Systems and Technology (TIST), 1(2), 1-24.
[11] Siciliano, B., & Khatib, O. (2016). Robotics and the handbook. In Springer Handbook of Robotics (pp. 1-6). Cham: Springer International Publishing.
[12] Slotine, J. J. E., & Li, W. (1991). Applied nonlinear control (Vol. 199, No. 1, p. 705). Englewood Cliffs, NJ: Prentice hall.
[13] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational physics, 378, 686-707.
[14] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. Nature Reviews Physics, 3(6), 422-440.
[15] Chen, R. T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural ordinary differential equations. Advances in neural information processing systems, 31.
[16] Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. Proceedings of the national academy of sciences, 113(15), 3932-3937.
[17] Liu, J., Borja, P., & Della Santina, C. (2024). Physics-informed neural networks to model and control robots: A theoretical and experimental investigation. Advanced Intelligent Systems, 6(5), 2300385.
[18] Li, H., Chai, Y., Lv, B., Ruan, L., Zhao, H., Zhao, Y., & Luo, J. (2025). Physics-informed Neural Network Predictive Control for Quadruped Locomotion. arXiv preprint arXiv:2503.06995.
[19] Li, Z., Wu, S., Chen, W., & Sun, F. (2025). Physics-informed neural networks for compliant robotic manipulators dynamic modeling. Journal of Computational Science, 90, 102633.
[20] Banerjee, C., Nguyen, K., Fookes, C., & Raissi, M. (2025). A survey on physics informed reinforcement learning: Review and open problems. Expert Systems with Applications, 287, 128166.
[21] Asgharpoor Golroudbari, A. (2024). TE-PINN: Quaternion-Based Orientation Estimation using Transformer-Enhanced Physics-Informed Neural Networks. arXiv e-prints, arXiv-2409.
[22] Tao, F., Zhang, M., & Nee, A. Y. C. (2019). Digital twin driven smart manufacturing. Academic press.
[23] Grieves, M., & Vickers, J. (2016). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In Transdisciplinary perspectives on complex systems: New findings and approaches (pp. 85-113). Cham: Springer International Publishing.
[24] Mohaghegh, N., Wang, H., & Yazdani, A. (2025). Sim2Real Transfer of Imitation Learning of Motion Control for Car-like Mobile Robots Using Digital Twin Testbed. Robotics, 14(12), 180.
[25] Liu, D., Chen, Y., & Wu, Z. (2023). Digital twin (dt)-cyclegan: Enabling zero-shot sim-to-real transfer of visual grasping models. IEEE Robotics and Automation Letters, 8(5), 2421-2428.
[26] Wang, S., Tao, J., Jiang, Q., Chen, W., Qin, C., & Liu, C. (2024). A digital twin framework for anomaly detection in industrial robot system based on multiple physics-informed hybrid convolutional autoencoder. Journal of Manufacturing Systems, 77, 798-809.
[27] Datta, G., Sharif, S. S., & Banad, Y. M. (2026). Toward Digital Twins in 3D IC Packaging: A Critical Review of Physics, Data, and Hybrid Architectures. arXiv preprint arXiv:2601.23226.
[28] Fu, T., Hu, Z., Zhang, T., Bi, Q., & Song, X. (2024). Physics-informed neural networks-based online excavation trajectory planning for unmanned excavator. Chinese Journal of Mechanical Engineering, 37(1), 131.
[29] Ni, R., & Qureshi, A. H. (2024, May). Physics-informed neural motion planning on constraint manifolds. In 2024 IEEE International Conference on Robotics and Automation (ICRA) (pp. 12179-12185). IEEE.

ARTICLE IN PRESS